

Ex. No.: 9
Date: 4/4/25

DEADLOCK AVOIDANCE

Aim:

To find out a safe sequence using Banker's algorithm for deadlock avoidance.

Algorithm:

1. Initialize work=available and finish[i]=false for all values of i
2. Find an i such that both:
finish[i]=false and Need_i ≤ work
3. If no such i exists go to step 6
4. Compute work=work+allocation_i
5. Assign finish[i] to true and go to step 2
6. If finish[i]=true for all i, then print safe sequence
7. Else print there is no safe sequence

Program Code:

```
#include <stdio.h>
#include <stdbool.h>
#define MAX_PROCESSES 5
#define MAX_RESOURCES 3
bool is safe (int process[], int avail[], int max[],
              [MAX_PROCESSES],
              [MAX_RESOURCES]);
int allot[][MAX_PROCESSES][MAX_RESOURCES];
bool finish [MAX_PROCESSES] = [false];
int work [MAX_RESOURCES];
for (int i = 0; i < n; i++) {
    for (int j = 0; j < m; j++) {
        need[i][j] = max[i][j] - allot[i][j];
    }
}
for (int i = 0; i < m; i++) {
    work[i] = avail[i];
}
int safe sequence [MAX_PROCESSES];
int count = 0;
```

```
while (count < n){
```

```
    bool found = false;
```

```
    for (int i=0; i<n; i++){
```

```
        if (! finished[i]){
```

```
            int j;
```

```
            for (j=0; j<m; j++){
```

```
                if (need[i][j] > work[j])
```

```
                    break;
```

```
            }
```

```
            if (j==m){
```

```
                for (int k=0; k<m; k++){
```

```
                    work[k] += allot[i][k];
```

```
                }
```

```
                finish[i] = true;
```

```
                safe sequence [count++] = i;
```

```
                found = true;
```

```
                break;
```

```
            }
```

```
        }
```

```
    }
```

```
    printf ("safe sequence");
```

```
    for (int i=0; i<n; i++){
```

```
        printf ("p %d", safe sequence [i]);
```

```
    }
```

```
    printf ("\n");
```

```
    return true;
```

```
}
```

```
int main(){
```

```
    int processes [MAX_PROCESSES];
```

```
    int avail [MAX_RESOURCES];
```

```
    printf ("Enter available resources (ABC): ");
```

```
    for (i=0; i<MAX_RESOURCES; i++){
```

```
        scanf ("%d", &avail[i]);
```

```
    }
```

```

int max [MAX-PROCESSES][MAX-RESOURCES];
printf ("Enter max demand matrix /  

      Max resource per each process)\n");
for (int i=0 ; i < MAX-PROCESSES ; i++){
    printf ("Enter max demand matrix  

      ("Max resource per each process):\n");
    for (int i = 0 ; i < MAX-PROCESSES ; i++){
        printf ("Enter max demand for process")
    }
    if (!safe)
        for (int j=0 ; j < MAX-RESOURCES ; j++){
            scanf ("%d", &max[i][j]);
        }
    int allot [MAX-PROCESSES][MAX-RESOURCES];
    printf ("Enter allocation matrix (  

      resources allocated to each  

      process):\n");
    for (int i=0 ; i < MAX-PROCESSES ; i++){
        printf ("Enter allocation for  

      process P %d ", i);
        for (int j=0 ; j < MAX-RESOURCES ; j++){
            scanf ("%d", &allot[i][j]);
        }
    }
    if (!safe (procen, avail, man, allot,  

      n, m))
        return 0;
    return 0;
}
return 0;

```

OUTPUT :

Enter no. of process & resource : 5 3

Enter alloc of resource of all process :

0	1	0
2	0	0
3	0	2
2	1	1

Enter max resource required :

0	0	2
---	---	---

7 5 3

3 2 2

9 0 2

4 2 2

5 8 3

Enter Available resource : 3 3 2

Sample Output:

The SAFE Sequence is
P1 → P3 → P4 → P0 → P2

Need Resource Matrix :

7 4 3

1 2 2

6 0 0

2 1 1

5 3 1

Available resource after completion : 10 5 7

safe seq are : P1 → P3 → P4 → P0 → P2

Result:

Hence safe sequence is obtained for deadlock
avoidance using bankers algorithm .

OK